The Oracle Separation of BQP and PH

Mohamed El Mandouh Sidhant Saraogi Sebastian R. Verschoor

July 2019

1 Introduction

In May 2018 Ran Raz and Avishay Tal excited the quantum complexity community by showing that there exists an oracle O relative to which $\mathsf{BQP}^O \not\subseteq \mathsf{PH}^O$. Although this oracle separation has limited consequences in the real world[1], the importance of this result has been argued by others more eloquent than we are able to, so we refer to them for motivation of this work [Aar09, Aar18, BB18, For18]. In this report we attempt to discuss the paper in some detail.

The proof by Raz and Tal shows that there exists a distribution (a variant of the Forrelation distribution) that a quantum algorithm can distinguish from the uniform distribution effectively, whereas no sub-exponentially sized Boolean circuit of constant depth is able to do so. We will make this vague statement more precise later. Using standard amplification techniques, the quantum advantage can be amplified to show a super-polynomial gap between the power of BQP machines and constant depth circuits in the black-box model. The oracle separation is then implied via standard relations—both those between AC^0 and PH and the relation between black-box separations and oracle separations. Despite these techniques and relations being standard, we consider them non-trivial and will elaborate on how the main result follows from the proven theorems.

A critical part of the argument [RT18] for the oracle separation of BQP and PH is to consider instead of just one choice of an oracle O, but some *distribution* $\mathcal{D}$ over pairs of oracles, which can be converted into a standard oracle such that $\mathsf{BQP}^O \not\subseteq \mathsf{PH}^O$, as shown in section 5. The main result of the paper is that there exists a (log time) quantum algorithm that distinguishes the Forrelation distribution from the uniform distribution with some advantage—which can be boosted. In addition, no constant-depth circuit can distinguish between the Forrelation and uniform distribution with the necessary advantage. This implies that no machine M in PH is able to distinguish between the two distributions effectively. Consider the distribution $\mathcal{D}$ (as defined later) over the inputs $\{\pm 1\}^{2N}$. Which can be thought of as a distribution over two oracles $x, y : \{0,1\}^N \longrightarrow \{\pm 1\}$. The theorem from which the main result follows is:

Theorem 1 (Main Theorem)**.** *There is a quantum algorithm that makes one query to the input, and runs in time $O(\log N)$, that distinguishes between $\mathcal{D}$ and the uniform distribution with advantage $\Omega(1/\log N)$. In addition, no Boolean circuit of size* $\mathsf{quasipoly}(N)$ *and constant depth distinguishes between $\mathcal{D}$ and the uniform distribution with advantage better than* $\mathsf{polylog}(N)/\sqrt{N}$.

The advantage of the quantum algorithm can be amplified by making $\mathsf{polylog}(N)$ sequential repetitions. Intuitively this means that there is a quantum algorithm that makes one quantum query to each of the oracles X and Y and distinguishes between the two distributions they were sampled from with an advantage of at least $1/\mathsf{poly}(N)$. Our goal then is to show that distinguishing $\mathcal{D}$ from the uniform distribution is easy for a quantum computer, and hard for a PH–machine. In other words, we need to find $\mathcal{D}$ that appears pseudo–random for AC^0 but not pseudo–random for a quantum polylog time algorithm.

As with these kind of problems the quantum algorithm is easy, while the classical lower bound is the challenging part of the proof. We will begin by introducing all the necessary ingredients to construct the distribution $\mathcal{D}$, the quantum algorithm, and finally the classical circuit lower bound.

2 Background

To simplify the analysis it is convenient to label Boolean values as $\{\pm 1\}$ instead of the more conventional $\{0,1\}$. Let N be the input length under the restriction that $N = 2^n$ for some large enough integer n.

We use an idea often used in cryptography: we say that a decision algorithm A *distinguishes* between distributions $\mathcal{D}$ and $\mathcal{D}'$ with *advantage* α if $|\Pr_{x\sim\mathcal{D}}[A \text{ accepts } x] - \Pr_{x'\sim\mathcal{D}'}[A \text{ accepts } x']| = \alpha$. Note that for a decision algorithm

[1] The result implies that $\mathsf{Promise-BQLOGTIME} \not\subseteq \mathsf{Promise-AC}^0$ but nothing is proven about BQP versus PH.

$\mathsf{A} : \{\pm 1\}^N \to \{\pm 1\}$ we have $|\mathbf{E}_{x\sim\mathcal{D}}[A(x)] - \mathbf{E}_{x'\sim\mathcal{D}'}[A(x')]| = 2\alpha$, although the constant difference can be ignored in the asymptotic results.

Let $\mathcal{N}(0,\varepsilon)$ denote the Gaussian distribution with mean 0 and variance ε. We require the standard bound $\Pr[|\mathcal{N}(0,\varepsilon)| \geq x] \leq \exp(-x^2/(2\varepsilon))$.

In order to analyze a Boolean circuit $A : \{\pm 1\}^{2N} \to \{\pm 1\}$, we need to the unique multi–linear extension $A : \mathbb{R}^{2N} \to \mathbb{R}$, which can be written as the polynomial

$$A(z) = \sum_{S\subseteq[2N]} \hat{A}(S) \cdot \prod_{i\in S} z_i, \tag{1}$$

where $\hat{A}(S)$ are the Fourier coefficients of A:

$$\hat{A}(S) = \left\langle A, \prod_{i\in S} z_i \right\rangle = \mathop{\mathbf{E}}_{x}\left[A(z) \cdot \prod_{i\in S} z_i\right]. \tag{2}$$

The reason this is useful is that $\mathbf{E}_{z'\sim\mathcal{D}}[A(z')] = \mathbf{E}_{z\sim\mathcal{G}'}[A(\mathrm{trnc}(z))]$ for any multilinear function that maps $[-1,1]^{2N}$ to $[-1,1]$ and in fact $\mathbf{E}_{z'\sim\mathcal{D}}[A(z')] \approx \mathbf{E}_{z\sim\mathcal{G}'}[A(z)]$, where the difference introduced by truncation can mostly be ignored in the analysis since truncation happens with only negligible probability. We note that this is formally proven [RT18], but we omit the proof due to space concerns.

2.1 The Polynomial Hierarchy and BQP

The complexity classes P and NP are well known. P is the set of problems that can be solved *efficiently* and deterministically, while NP is the set of problems for which there are efficiently *verifiable* solutions. It is useful to define these more rigorously.

Definition 1 (P)**.** A language L is in P if and only if there exists a polynomial time uniform family of Boolean circuits $\{C_n \ : n \in \mathbb{N}\}$, such that

1. For all $n \in \mathbb{N}$, C_n takes n bits as input and outputs 1 bit
2. For all x in L, $C_{|x|}(x) = 1$
3. For all x not in L, $C_{|x|}(x) = 0$

And we can similarly define NP as follows.

Definition 2 (NP)**.** A language L is in NP if and only if there exists polynomials p and q, and a deterministic Turing machine V, such that

1. $\forall x, y$, the machine V runs in time $p(|x|)$ on input (x, y)
2. $\forall x \in \mathsf{L}$, there exists a string y of length $q(|x|)$ such that $V(x, y) = 1$
3. $\forall x \notin \mathsf{L}$ and all strings y of length $q(|x|)$: $V(x, y) = 0$

So NP is the class of decision problems, such that given a candidate answer and a proof we can verify the correctness of the proof efficiently (in polynomial time in the size of the input). We then simply define coNP as the class of decision problems that can efficiently verify *counter examples*. It is then natural to wonder how we can generalize the notions of P, NP, and coNP. The Polynomial-Time Hierarchy is defined as follows [Sto76]:

Definition 3 (PH)**.** Let

- $\Sigma_0^p = \mathsf{P}$
- $\Sigma_1^p = \mathsf{NP}$
- $\Sigma_{k+1}^p = \mathsf{NP}^{\Sigma_k^p}$

where NP^{A} is the class of problems solvable in non–deterministic polynomial time with access to an oracle for solving problems in A. Then the union $\bigcup_k \Sigma_k^p = \mathsf{PH}$ forms the Polynomial–Time Hierarchy.

An equivalent definition of Σ_i^p defines the languages L it contains.

Definition 4 (PH (alternative))**.** $\mathsf{L} \in \Sigma_i^p$ if there exists a polynomial-time solvable Boolean formula ϕ such that

$$x \in \mathsf{L} \Leftrightarrow \exists y_1 \forall y_2 \ldots Q_i y_i : \phi(x, y_1, y_2, \ldots, y_i) = 1, \tag{3}$$

where $|y_j| = \mathsf{poly}(|x|)$ for all $j \leq i$ and Q_i denotes $\forall$ or $\exists$ if i is even or odd respectively. Then $\mathsf{PH} = \bigcup_i \Sigma_i^p$.

A natural generalization is to consider when the first quantifier is $\forall$. This defines Π_i^p and it is the complement to Σ_i^p. Note that $\Pi_i^p \subseteq \Sigma_{i+1}^p$, so for our purposes it suffices to consider only definition 4.

We are of course now eager to define the class of problems that are efficiently solvable by a quantum computer.

Definition 5 (BQP)**.** A language L is in BQP if and only if there exists a polynomial time uniform family of quantum circuits $\{Q_n \ : n \in \mathbb{N}\}$, such that

1. For all $n \in \mathbb{N}$, Q_n takes n qubits as input and outputs 1 bit
2. For all x in L, $\Pr(Q_{|x|}(x) = 1) \geq 2/3$
3. For all x not in L, $\Pr(Q_{|x|}(x) = 0) \geq 2/3$

We can immediately see that $\mathsf{P} \subseteq \mathsf{BQP}$. While BQP is not exactly analogous to P, it is analogous to BPP, the class of problems solved efficiently by a probabilistic Turing machine. However, it has been shown [Sip83, Lau83] that BPP is contained in PH.

Surprisingly, the main result of the paper does not explicitly depend on BQP or PH. It instead establishes a new upper bound on the effectiveness of circuits with constant depth.

2.2 AC^0 and PH

Definition 6 (The AC hierarchy)**.** Let AC be the union of all the classes AC^k. A circuit $C : \{0,1\}^n \to \{0,1\}$ is in AC^k if it has size $\mathsf{poly}(n)$ and depth $O(\log^k(n))$ and has unbounded fan–in AND, OR, and NOT gates.

Thus AC^0 is the smallest class of the AC hierarchy, of depth $O(1)$ and unbounded fan–in AND, OR, and NOT gates. From here on we slightly relax the definition of an AC^0-*circuit* to mean any constant-depth circuit with unbounded fan-in (but not necessarily of polynomial size).

We can simplify any AC^0-circuit significantly by using two tricks. First use De Morgan's laws to propagate any NOT-gate to the input leaves (for example: $\mathrm{NOT}(\mathrm{AND}(x, y)) = \mathrm{OR}(\mathrm{NOT}(x), \mathrm{NOT}(y))$, which gives an equivalent circuit without NOT-gates and access to both input variables and their negation. Second merge any consecutive similar gates (for example: $\mathrm{AND}(x, \mathrm{AND}(y, z)) = \mathrm{AND}(x, y, z)$), which we are allowed to do because the fan-in is not bounded. Note that neither of the transformations increases the circuit depth. What remains is a circuit with alternating AND/OR gates.

This simplified circuit highlights the relation between AC^0 and PH, as observed by Furst, Saxe and Sipser [FSS81]. Given a PH machine M solving some decision problem, reinterpret the existential quantifiers of M as OR gates and the universal quantifiers as AND gates. If M is a Σ_d^p machine, then this will give an AC^0-circuit of depth d and size $\mathsf{poly}(n)$. This means that given a machine M in PH with access to some oracle O, M can be simulated by circuit C in AC^0 if C has access to O.

2.3 The Hadamard Transform

Let $n \in \mathbb{N}$ and $N = 2^n$. Then we define the Hadamard transform $H = H_N \in \mathbb{R}^{N \times N}$ as follows:

$$H_{ij} = \frac{1}{\sqrt{N}} \cdot (-1)^{\langle i,j \rangle} \tag{4}$$

for $i, j \in [N]$ and $\langle i, j \rangle$ is the dot product of the binary representations of $(i-1)$ and $(j-1)$. We also remark that H_N is orthogonal and symmetric and thus $H_N^2 = I_N$.

2.4 A Variant of the Forrelation Distribution

The distribution $\mathcal{D}$ that the authors use is a variation of the so called Forrelation distribution first introduced by Aaronson [Aar09]. The main idea of the distribution is to choose the oracle y to be correlated to the Fourier transform of x. We sample x from a normal distribution with mean 0 and variance 1, $x_1, x_2, \ldots, x_N$ where $N = 2^n$, and n the size of inputs to the oracle x. Then we let $y = H_N x$. Then multiply each of the values in x and y with some ε, which the authors chose to be $1/(24 \ln N)$. Next, we take the values in εx and εy and *probabilistically* round them to -1 and

1 in the following manner: First, we draw $z \in (\varepsilon x, \varepsilon y)$ and take $\mathrm{trnc}(z) := (\mathrm{trnc}(z_1), \mathrm{trnc}(z_2), \ldots, \mathrm{trnc}(z_{2N}))$, where $\mathrm{trnc}(a) := \min(1, \max(-1, a))$. Then independently for each $i \in [2N]$, we draw $z'_i = 1$ with probability $\frac{1+\mathrm{trnc}(z_i)}{2}$ and $z'_i = -1$ with probability $\frac{1-\mathrm{trnc}(z_i)}{2}$ and output $z' \in \{\pm 1\}^{2N}$. This final output is the what the authors denote $\mathcal{D}$.

Both the Forrelation distribution $\mathcal{F}$ and the variant $\mathcal{D}$ over $\{\pm 1\}^N \times \{\pm\}^N$ have the distribution $\mathcal{G}$ over $\mathbb{R}^N \times \mathbb{R}^N$ in their construction. $\mathcal{G}$ is sampled as

1. Sample $x_1, x_2, \ldots, x_N \sim \mathcal{N}(0, 1)$ independently
2. Let $x = (x_1, x_2, \ldots x_N)$ and $y = H_N \cdot x$
3. Output $z = (x, y)$

Forrelation then maps this directly to a Boolean vector $\mathrm{sgn}(z)$ (apply the sign-function pointwise). The variant instead first maps to distribution $\mathcal{G}'$ by multiplying the output pointwise by $\sqrt{\varepsilon}$, where $\varepsilon = 1/(24 \ln N)$. We get $z' \sim \mathcal{D}$ by *probabilistically* rounding the points to ± 1: define $\mathrm{trnc}(a) := \min(1, \max(-1, a))$. Independently for each $i \in [2N]$, draw $z'_i = 1$ with probability $\frac{1+\mathrm{trnc}(z_i)}{2}$ and $z'_i = -1$ with probability $\frac{1-\mathrm{trnc}(z_i)}{2}$ and output z'.

The heart of the problem lies in the distribution $\mathcal{G}$, the rest of the construction just transforms $\mathcal{G}$ into a Boolean distribution in such a way that the proof works. We make some useful observations on $\mathcal{G}$. Note that $\mathcal{G}$ is a multivariate Gaussian distribution with mean 0 and covariance matrix $\begin{bmatrix} I_N & H_N \\ H_N & I_N \end{bmatrix}$.

Note that $\mathcal{G}$ isn't very different from the original Forrelated distribution $\mathcal{F}$ described in Aaronson's paper. In fact, we can prove (as provided by a detailed argument in [RT18]) that:

Lemma 1 (Eqn(2) in [RT18])**.** *Let $A : \mathbb{R}^{2N} \to \mathbb{R}$ be a multilinear function that maps $[-1, 1]^{2N}$ to $[-1, 1]$ where A is the multilinear extension of the Boolean function as in Definition 2.1. Then,*

$$\mathop{\mathbf{E}}_{z' \sim D}[A(z')] = \mathop{\mathbf{E}}_{z \sim \mathcal{G}'}[A(\mathrm{trnc}(z))]$$

Proof. Write A as eq. (1) and use the linearity of expectation:

$$\mathbf{E}[A(z')|z] = \mathbf{E}\left[\sum_{S \subseteq [2N]} \hat{A}(S) \prod_{i \in S} z'_i | z\right] = \sum_{S \subseteq [2N]} \hat{A}(S) \prod_{i \in S} \mathbf{E}[z'_i | z] = \sum_{S \subseteq [2N]} \hat{A}(S) \prod_{i \in S} \mathrm{trnc}(z_i) = A(\mathrm{trnc}(z))$$

□

2.5 Bounds on Moments

We want to bound the "moments" of $\mathcal{G}$ defined on sets of indices $S, T \subseteq [N]$. Recall that the moments of G are simply the expectation of the Fourier characters over the underlying distribution.

$$\hat{\mathcal{G}}(S, T) \triangleq \mathop{\mathbf{E}}_{(x,y) \sim \mathcal{G}}\left[\prod_{i \in S} x_i \cdot \prod_{j \in T} y_i\right] \tag{5}$$

In general a bound is given by $|\hat{\mathcal{G}}(S, T)| \leq 1$, following from Cauchy-Schwarz and the linearity of expectation, but a better bound exists for some coefficients:

Claim 1.1 (Claim 4.1 [RT18])**.** *Let $S, T \subseteq [N]$, then*

$$|\hat{\mathcal{G}}(S, T)| \begin{cases} = 0 & \text{if } |S| \neq |T| \\ \leq k! N^{-k/2} & \text{if } |S| = |T| = k \end{cases}$$

Proof. Isserlis' Theorem [Iss18] expresses expected value of a product of multivariate Gaussians as a sum of products of pairwise expected value. The theorem states that for distinct $i_1, i_2, \ldots, i_2 k \in [2N]$ $\mathbf{E}[Z_{i_1} \ldots Z_{i_{2k-1}}] = 0$ and $\mathbf{E}[Z_{i_1} \ldots Z_{i_{2k}}] = \sum \prod \mathbf{E}[Z_{i_l} Z_{i_r}]$, where $\sum \prod$ is the sum over all ways $Z_{i_1}, \ldots, Z_{i_{2k}}$ can be partitioned into pairs. If a pair has entries from the same half, the corresponding random variables are independent and thus $\mathbf{E}[Z_{i_l} Z_{i_r}] = 0$. When $|S| \neq |T|$ every term of the sum contains such a pair. Otherwise, there are $k!$ partitions such that no pairs come from the same half. For every such pair the covariance $\pm N^{-1/2}$ is given directly by the covariance matrix. With k pairs per product and $k!$ partitions we get $\hat{\mathcal{G}}(S, T) \leq k! N^{-k/2}$. □

Note that a similar bound holds for $\mathcal{G}'$, but with covariance matrix $\varepsilon \operatorname{cov}(\mathcal{G})$.

3 The Quantum Algorithm

The quantum algorithm for distinguishing $\mathcal{D}$ and the uniform distribution follows from the circuit proposed by Aaronson and Ambianis for solving the Forrelation problem [AA14]. Let Q be the 1–query algorithm for Forrelation. Then on a given input $x \in \{\pm 1\}^N$ and $y \in \{\pm 1\}^N$, the algorithm Q accepts with probability $(1 + \varphi(x, y))/2$, where

$$\varphi(x, y) := \frac{1}{N} \cdot \sum_{i \in [N], j \in [N]} x_i \cdot H_{i,j} \cdot y_j. \tag{6}$$

Then the following claims are true:

1. $\mathbf{E}_{(x,y) \sim U_{2N}}[\varphi(x, y)] = 0$
2. $\mathbf{E}_{(x,y) \sim \mathcal{D}}[\varphi(x, y)] \geq \varepsilon/2$

This means that if x and y are sampled from the uniform distribution then the $\mathbf{E}(\varphi(x, y)) = 0$. On the other hand, if x and y are sampled from $\mathcal{D}$ then the expected value is $\mathbf{E}(\varphi(x, y)) \geq \varepsilon/2$. Thus the quantum algorithm can distinguish $\mathcal{D}$ from the uniform distribution with advantage $\varepsilon = \Omega(\frac{1}{\log(N)})$, which can be amplified to distinguish in poly–time with advantage $1 - 2^{-\mathsf{polylog}(N)}$. More precisely, let $\mathcal{D}_1 = \mathcal{D}^{\otimes m}$ be the concatenation of m independent random variables with distribution $\mathcal{D}$. Then we run the quantum algorithm independently on each random variable and sum the outcomes. We accept iff the sum is greater or equal to $m\varepsilon/4$. This gives an algorithm with run–time $O(m \log N)$. Setting $m = 32 \ln(1/\delta)/\varepsilon^2$ for some $\delta \geq 2^{-\mathsf{polylog}(N)}$ the run–time is poly-logarithmic. By Chrenoff's bound the algorithm (wrongly) accepts the uniform distribution with probability at most $\exp[-m(\varepsilon/4)^2/2] \leq \delta$, while accepting $\mathcal{D}$ with probability at least $1 - \exp[-m(\varepsilon/4)^2/2] \geq 1 - \delta$. This proves the first statement of theorem 1.

The proof for the claims is given in the main paper [RT18], we seek an intuitive explanation for why the quantum algorithm works. The circuit is composed of 1 control qubit and N input qubits. The quantum algorithm queries x and y in superposition, and prepares the following state

$$\sum_{i \in [N]} x_i |0\rangle|i\rangle + \sum_{i \in [N]} y_i |1\rangle|i\rangle. \tag{7}$$

In other words, we output $\sum_{i \in [N]} x_i |i\rangle$ and $\sum_{i \in [N]} y_i |i\rangle$ if we measure $|0\rangle$ and $|1\rangle$ respectively. We can then efficiently apply the Hadamard transform H_N (in $O(\log(N))$ gates) on the second half of the input qubits. Now remember that $y = H_N x$ and $H_N^2 = I$, then $H_N y = x$ and the control qubit is unentangled from the input. In addition, y is ε correlated with x so if we then apply a final Hadamard on the control qubit, we measure $|0\rangle$ with ε more probability than $|1\rangle$ (as given by (6)). But, if they are sampled from the uniform distribution then we measure the control qubit with equal probabilities.

4 Lower Bound for AC^0

It is helpful to take a moment and think about why proving $\mathsf{BQP} \not\subseteq \mathsf{PH}$ was difficult. All known techniques for proving a circuit is not in AC^0 is by showing that it cannot be approximated by a low–degree polynomial. For example, we have the following lemma:

Lemma 2 (Razborov's Lemma)**.** *For all AC^0 circuits of size s, depth d, then there exists a distribution of polynomials $p(x_1, x_2, \ldots, x_n) \in \mathbb{F}[X_1, X_2, \ldots, X_n]$ such that, for all x*

$$\Pr_{p \in P}[p(x) \neq C(x)] \leq \varepsilon, \tag{8}$$

holds for all finite fields $\mathbb{F}$ and $\deg(p) \leq \log(s/\varepsilon)^d$.

So to prove a circuit is not in AC^0 it is sufficient to show that it is not approximated by a low–degree polynomial over some field $\mathbb{F}$. The canonical example is the PARITY function. While we have an exact degree–1 polynomial over $\mathbb{F}_2$, $\sum_i^n x_i$, this is not enough. Since, it can be shown that over other finite fields $\mathbb{F}_n$ where $n \neq 2$ any low–degree polynomial disagrees with PARITY on a large fraction of the inputs, thus by the lemma $\mathsf{PARITY} \notin \mathsf{AC}^0$.

On the other hand, every function with a low quantum query complexity is approximated by a low–degree polynomial.

Lemma 3 (Bounded Degree)**.** *If a quantum algorithm makes T queries to a black-box X. Then Q's acceptance probability is a real multi–linear polynomial $p(x)$, of degree at most $2T$.*

Proof. The polynomial method, proven in class. □

If our techniques rely on proving there is not some low–degree polynomial approximation, but every low quantum query complexity algorithm necessarily admits a low–degree polynomial then what can we do? What Tal showed earlier in [Tal17] is that not only are AC^0-circuits approximated by low–degree polynomials but their coefficients must also have bounded L_1 norms.

We can write any AC^0 circuit $A : \{\pm 1\}^{2N} \to \{\pm 1\}$ as a multi-linear polynomial on the inputs to the function represented by the circuit:

$$A(z) = \sum_{S \subseteq [2N]} \hat{A}(S) \cdot \prod_{i \in S} z_i.$$

Note that this representation is simply derived from observing the Fourier Expansion of the Boolean function represented by the circuit. From this definition and eq. (2) it immediately follows that $A(\vec{0}) = \hat{A}(\emptyset) = \mathbf{E}_{x \sim U_{2N}}[A(x)]$, encoding the overall bias of A.

However, in our case, we want to consider the value of $A(z)$ evaluated on a sample $z \sim \mathcal{G}'$.

Then, note that:

$$\mathop{\mathbf{E}}_{z \in \mathcal{G}'}[A(z)] = \sum_{S \subseteq [2N]} \hat{A}(S) \cdot \mathop{\mathbf{E}}_{z \in \mathcal{G}'} \left[\prod_{i \in S} z_i \right]$$

Theorem 2. ***(Tal's Bound [Tal17, RT18])*** *Let $A : \{\pm 1\}^{2N} \longrightarrow \{\pm 1\}$ be a Boolean circuit of a quasi–polynomial and constant depth. Then for a vector $z \in \mathbb{R}^{2N}$, we denote by $A(z)$ the value of the multi–linear extension of A on z. We can write $A(z) = \sum_{S \subseteq [2N]} \hat{A}(S) \cdot \prod_{i \in S} z_i$, where $\hat{A}(S)$ are the Fourier coefficients of A. Then we have the following bound on the Fourier coefficients,*

$$\sum_{S \subseteq [2N] : |S| = k} |\hat{A}(S)| \leq (\mathsf{polylog}(N))^k. \tag{9}$$

Tal's bound serves as the basis for all the bounds in the following proof. However this is not enough, we will need the random walk argument from the paper to take this further. For $k \leq \sqrt{N}$, Tal's bound is sufficient as the upper bound needed for our Theorem. However, the higher degree terms still might compensate. Therefore, we need a tighter upper bound on our bias polynomial.

Suppose instead of sampling $z \sim \mathcal{G}'$, we sample $z_1, \ldots, z_t \sim \mathcal{G}'$ and set $z = \frac{p}{\sqrt{t}}(z_1 + \cdots + z_t)$ $(p < 1/2)$. Note that this still implies that $z \sim \mathcal{G}'$. Let $Z_i = \frac{p}{\sqrt{t}}(z_1 + \cdots + z_i)$. Note $Z_t = z$.

The idea is to interpret the Gaussian as the sum of a large number of independent small Gaussians $(\frac{z_i}{\sqrt{t}})$. We will see that each smaller Gaussian contributes a sufficiently bounded bias to the final bias polynomial thereby proving our bound.

First, we bound Z_1's bias using Tal's bound.

Lemma 4.

$$\left| \mathop{\mathbf{E}}_{z_1 \sim \mathcal{G}'}[A(Z_1) - A(\vec{0})] \right| \leq O\left(\frac{\epsilon}{t}\right) polylog(N)$$

Proof.

$$\begin{aligned}
\left| \mathop{\mathbf{E}}_{z \sim \mathcal{G}'}[A\left(\frac{z_1}{\sqrt{t}}\right) - A(\vec{0})] \right| &= \left| \sum_{k=1}^{N} \sum_{S \subseteq [2N], |S| = 2k} \hat{A}(S) \mathop{\mathbf{E}}_{z_1 \sim \mathcal{G}'} \left[\prod_{i \in S} \frac{1}{\sqrt{t}} z_i \right] \right| \\
&\leq \sum_{k=1}^{N} \sum_{S \subseteq [2N], |S| = 2k} |\hat{A}(S)| O\left(\frac{\varepsilon k}{t}\right)^k && \text{(Iserliss Theorem)} \\
&\leq \sum_{k=1}^{N} (\mathsf{polylog}(N))^{2k} O\left(\frac{\varepsilon k}{t}\right)^k && \text{(Tal's Bound)} \\
&\leq O\left(\frac{\varepsilon}{t}\right) polylog(N) && \text{(Split between } k \leq N/2 \text{ and } k > N/2)
\end{aligned}$$

The last part inequality follows from splitting the sum between $k \leq N/2$ and $k > N/2$. □

We can consider the above as a base case for our proof. The crux of Raz and Tal's bound however lies in the Lemma below.

Lemma 5. *Let $z_0 \in [-1/2, 1/2]^{2N}$. Then, for small enough $p = \frac{1}{\sqrt{t}}$,*

$$\left| \mathop{\mathbf{E}}_{z \in \mathcal{G}'} [A(z_0 + p \cdot z) - A(z_0)] \right| \leq O\left(\frac{\varepsilon}{t}\right) \ \mathsf{polylog}(N)$$

Proof sketch. The proof for the above lemma first appeared (in a different form) in [CHHL18]. It relies on the idea that restrictions of a Boolean circuit (in this case A) are also Boolean circuits of the same size and depth. Restrictions can be viewed as fixing certain variables of the Boolean function to take a fixed value ± 1. The proof works by choosing a clever distribution R_{z_0} over the restriction space and finding a function $\tilde{z}(\rho)$ such that $\mathbf{E}_{\rho \sim R_{z_0}}[A(\tilde{z})] = A(z_0 + p \cdot z)$ and such that the fourier coefficients of $\tilde{z}(\rho)$ are exactly the fourier coefficients of $P \circ z$ under the restriction ρ. The lemma then follows by applying on Tal's bound on this restricted function. □

Now, Lemma 5 allow us to prove a simple upper bound on the increase in bias of the distinguishing polynomial when we move from step Z_i to Z_{i+1}.

Corollary 2.1.

$$|\mathop{\mathbf{E}}_{z_{i+1} \in \mathcal{G}'} [A(Z_{i+1}) - A(Z_i)]| \leq O\left(\frac{\varepsilon}{t}\right) \ \mathsf{polylog}(N)$$

Proof. Since, $(Z_i)_j \sim \mathcal{N}(0, \varepsilon)$, therefore $\Pr_{Z_i \in [-1/2.1/2]^{2N}}$ with high probability. Therefore, we can apply Lemma 5 since $Z_{i+1} = Z_i + p z_{i+1}$ to obtain our result. □

Theorem 3. *Let $A : \{\pm 1\}^{2N} \to \{\pm 1\}$ be a Boolean circuit of size s and depth d. Then,*

$$\left| \mathop{\mathbf{E}}_{z \sim \mathcal{D}} A(z) - \mathop{\mathbf{E}}_{z \sim U} A(z) \right| \leq \varepsilon \cdot \mathsf{polylog}(N)$$

Proof. We can sum over each of the bounds obtained for $i = 0, \ldots, t-1$ using the triangle inequality to obtain:

$$\begin{aligned}
\left| \mathop{\mathbf{E}}_{z \sim \mathcal{D}} A(z) - \mathop{\mathbf{E}}_{z \sim U} A(z) \right| &\simeq \left| \mathop{\mathbf{E}}_{z \sim \mathcal{G}'} A(z) - A(\vec{0}) \right| \\
&= \left| \mathop{\mathbf{E}}_{z_1, \ldots, z_t \sim \mathcal{G}'} A(Z_t) - A(\vec{0}) \right| \\
&\leq \sum_{i=0}^{t-1} \left| \mathop{\mathbf{E}}_{z_{i+1} \sim \mathcal{G}'} A(Z_{i+1}) - A(Z_i) \right| \\
&\leq \sum_{i=0}^{t-1} O(\epsilon/t) \cdot \mathsf{polylog}(N) \\
&\leq \varepsilon \cdot \mathsf{polylog}(N)
\end{aligned}$$

Note in the proof we assume $A(Z_0) = A(\vec{0})$.

□

5 Constructing an Oracle Separation

Objective : Construct an Oracle O from the distribution D such that $\mathsf{BQP}^O \not\subseteq \mathsf{PH}^O$.

We follow the argument from [FSUV12]. For any $N \in \mathbb{N}$, let $x_n \sim U_N$ with probability 1/2 and let $x_n \sim \mathcal{D}$ with probability 1/2. Let L be the unary language that consists of all 1^n such that x_n was drawn from $\mathcal{D}$.

1. Show that there exists a BQP^O machine M that decides L on all but finitely many value of n.

   We have a BQP machine Q which runs in polynomial time that :

   (a) If $1^n \in \mathsf{L}$, then x_n was sampled from D and Q accepts with probability at least $1 - 1/n^2$.

   (b) If $1^n \notin \mathsf{L}$, then x_n was sampled from U_n and Q rejects with probability at least $1 - 1/n^2$.

   Let $M = Q$. We can choose a large enough n_0 such that the probability that for all $n \geq n_0$, the machine M succeeds over over a distribution of oracles is $\Pi_{n \geq n_0}(1 - 1/n^2) \geq 0.9$. Then, for at least half the oracles O (with prob at least 0.5 over the choice of oracles O, $\Pr_M[M^O$ decides 1^n $] \geq 0.8$.

2. Show that with probability at least 0.5 over choice of O, no PH^O decides correctly on 1^n for all $n \geq 1$.

   Consider any PH machine P. Note that each string of length n for a particular oracle O is chosen independently of the strings of other lengths. Therefore, $\Pr_O[P^O$ decides correctly on string of length $n] \leq 0.51$ by our main theorem 1. Then, Pr[A succeeds on strings of length n for each n] $= 0$. Therefore, no PH machine succeeds for any oracle O.

Now, obtain the oracle O where both events happen. "Hard–wire the values of L on 1^n for $n < n_0$ to M, making it a BQP^O machine that decides L correctly on 1^n for all $n \geq 1$."

6 Conclusion and Open Problems

The distribution $\mathcal{D}$ gives an oracle separation between BQP and PH. We could interpret this as evidence for a separation in the non-relativized world. An interesting take on this is that it could explain why we have not found many quantum algorithms. The phase estimation algorithm—the only algorithm which provides a non-controversial speedup—focuses on solving a problem that is widely believed to be NP-intermediate. Instead, it could be that most quantum algorithms achieve a speedup for problems outside the polynomial hierarchy. Of course, very few natural problems are known that lie outside this hierarchy, so the real issue may not be that we are bad at finding quantum algorithms but instead we are bad at finding really hard problems.

An interesting open question springing from this line of thought is whether there exists an oracle O such that $\mathsf{BQP}^O \not\subseteq \mathsf{PH}^O = \mathsf{P}^O = \mathsf{NP}^O$.[2]This would provide us with evidence that it makes sense to build quantum computers even if we find an efficient algorithm for solving SAT.

Another open question is whether we can generalize this result to Aaronson's Forrelation distribution $\mathcal{F}$, since the main result seems to come from the distribution $\mathcal{G}$ that underlies both $\mathcal{F}$ and $\mathcal{D}$. This seems non-trivial, as the used proof techniques do not apply to $\mathcal{F}$.

References

[AA14] Scott Aaronson and Andris Ambainis. Forrelation: A problem that optimally separates quantum from classical computing. *Electronic Colloquium on Computational Complexity (ECCC)*, 21:155, 2014.

[Aar09] Scott Aaronson. BQP and the polynomial hierarchy. *Electronic Colloquium on Computational Complexity (ECCC)*, 16:104, 2009.

[Aar18] Scott Aaronson. The relativized BQP vs. PH problem (1993-2018). https://www.scottaaronson.com/blog/?p=3827, 2018. Accessed: 2019-07-02.

[BB18] Boaz Barak and Jarosław Błasiok. On the Raz-Tal oracle separation of BQP and PH. https://windowsontheory.org/2018/06/17/on-the-raz-tal-oracle-separation-of-bqp-and-ph/, 2018. Accessed: 2019-07-02.

[CHHL18] Eshan Chattopadhyay, Pooya Hatami, Kaave Hosseini, and Shachar Lovett. Pseudorandom generators from polarizing random walks. In *33rd Computational Complexity Conference*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik, 2018.

[For18] Lance Fortnow. BQP not in the polynomial-time hierarchy in relativized worlds. https://blog.computationalcomplexity.org/2018/06/bqp-not-in-polynomial-time-hierarchy-in.html, 2018. Accessed: 2019-07-02.

[FSS81] Merrick L. Furst, James B. Saxe, and Michael Sipser. Parity, circuits, and the polynomial-time hierarchy. In *22nd Annual Symposium on Foundations of Computer Science*, pages 260–270. IEEE Computer Society, 1981.

[FSUV12] Bill Fefferman, Ronen Shaltiel, Christopher Umans, and Emanuele Viola. On beating the hybrid argument. In *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*, pages 468–483. ACM, 2012.

[2]In a presentation (http://www.birs.ca/events/2018/5-day-workshops/18w5197) Tal stated this question was answered affirmative by Aaronson and Fortnow, but we could not find this result.

[Iss18] Leon Isserlis. On a formula for the product-moment coefficient of any order of a normal frequency distribution in any number of variables. *Biometrika*, 12(1/2):134–139, 1918.

[Lau83] Clemens Lautemann. BPP and the polynomial hierarchy. *Information Processing Letters*, 17(4):215–217, 1983.

[RT18] Ran Raz and Avishay Tal. Oracle separation of BQP and PH. *Electronic Colloquium on Computational Complexity (ECCC)*, 25:107, 2018.

[Sip83] Michael Sipser. A complexity theoretic approach to randomness. In *Proceedings of the fifteenth annual ACM symposium on Theory of computing*, pages 330–335. Citeseer, 1983.

[Sto76] Larry J Stockmeyer. The polynomial-time hierarchy. *Theoretical Computer Science*, 3(1):1–22, 1976.

[Tal17] Avishay Tal. Tight bounds on the fourier spectrum of AC0. In *32nd Computational Complexity Conference (CCC 2017)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik, 2017.